

CA-112: PROGRAMMING USING C++

Deep Study Notes

FYBCA | KBCNMU University
Shri Seth Murlidharji College, Pachora

All Units | All Programs with Output | Exam Ready

UNIT 1: Basics of C++ — Tokens, Variables, Operators

1.1 Introduction to C++

C++ is an object-oriented programming language developed by Bjarne Stroustrup in 1979 at Bell Labs. It is an extension of the C language with additional features like classes, objects, inheritance, polymorphism, and more. C++ is used for system programming, game development, embedded systems, and application development.

Key Facts About C++

C++ = C language + Object Oriented Features
Developed by: Bjarne Stroustrup (1979, Bell Labs)
File Extension: .cpp
Compiler: g++ (GNU), Turbo C++, Dev C++
C++ is case-sensitive: 'int' and 'INT' are different

1.2 Tokens in C++

A Token is the smallest meaningful unit in a C++ program. Every statement in C++ is made up of tokens. The compiler processes these tokens one by one.

There are 5 types of tokens in C++:

- **Keywords** — Reserved words with special meaning (int, float, if, else, while, return, class, etc.)
- **Identifiers** — Names given by programmer to variables, functions, objects (age, total, student)
- **Constants / Literals** — Fixed values that do not change during program execution (10, 3.14, 'A', "Hello")
- **Operators** — Symbols that perform operations (+, -, *, /, %, ==, !=, &&, ||)
- **Punctuators / Special Symbols** — Symbols used for structure (,, {}, [], (), .., .)

Keywords in C++ (important list):

Keyword	Purpose	Example
int	Integer data type declaration	int age = 20;
float	Float data type declaration	float marks = 85.5;
char	Character data type	char grade = 'A';
bool	Boolean data type	bool passed = true;
if / else	Conditional statement	if(x>0){ } else{ }
for	Loop statement	for(i=0; i<5; i++)
while	While loop	while(n>0){ }
do	Do-while loop	do{ }while(x>0);
switch	Multi-branch selection	switch(day){ }

Keyword	Purpose	Example
return	Return value from function	return x+y;
void	No return type / no parameters	void show(){ }
class	Define a class (OOP)	class Student{ };
new	Dynamic memory allocation	int *p = new int;
delete	Free dynamic memory	delete p;
cout	Output stream (print)	cout << "Hello";
cin	Input stream (read)	cin >> age;

1.3 Variables in C++

A variable is a named storage location in memory that holds a value. The value stored in a variable can change during program execution. Every variable must be declared with a data type before it is used.

Syntax: `data_type variable_name = value;`

Data Type	Size	Range	Example
int	4 bytes	-2,147,483,648 to 2,147,483,647	int x = 10;
float	4 bytes	3.4e-38 to 3.4e+38 (6 decimals)	float f = 3.14;
double	8 bytes	1.7e-308 to 1.7e+308 (15 decimals)	double d = 9.99;
char	1 byte	-128 to 127 (or 0 to 255)	char c = 'A';
bool	1 byte	true (1) or false (0)	bool b = true;
long	8 bytes	Very large integers	long l = 99999L;

Rules for Naming Identifiers (Variables):

- Must start with a letter (a-z, A-Z) or underscore (_)
- Can contain letters, digits (0-9), and underscore
- Cannot use keywords as variable names (e.g. int, float are NOT valid names)
- Case-sensitive: 'Age' and 'age' are two different variables
- No spaces allowed in variable names
- No special characters except underscore (_)

1.4 Operators in C++

Operators are symbols that perform specific operations on operands (values or variables). C++ has a rich set of operators.

1.4.1 Arithmetic Operators:

Operator	Name	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division	10 / 3	3 (integer division)
%	Modulus (Remainder)	10 % 3	1
++	Increment	x++ or ++x	x = x + 1
--	Decrement	x-- or --x	x = x - 1

1.4.2 Relational Operators:

Operator	Meaning	Example	Result
==	Equal to	5 == 5	true (1)
!=	Not equal to	5 != 3	true (1)
>	Greater than	5 > 3	true (1)
<	Less than	3 < 5	true (1)
>=	Greater than or equal to	5 >= 5	true (1)
<=	Less than or equal to	3 <= 5	true (1)

1.4.3 Logical Operators:

Operator	Name	Example	Meaning
&&	Logical AND	a>0 && b>0	Both conditions must be true
	Logical OR	a>0 b>0	At least one must be true
!	Logical NOT	!(a>0)	Reverses the condition

1.4.4 Operator Precedence (Higher to Lower):

Priority	Operators	Associativity
1 (Highest)	() [] -> .	Left to Right
2	++ -- ! ~ (unary)	Right to Left
3	* / %	Left to Right
4	+ -	Left to Right
5	< > <= >=	Left to Right
6	== !=	Left to Right
7	&&	Left to Right
8		Left to Right
9 (Lowest)	= += -= *= /=	Right to Left

Program: Operators Example

```
// operators_example.cpp
#include<iostream>
using namespace std;
int main(){
    int a = 10, b = 3;
    cout << "a + b = " << a+b << endl;
    cout << "a - b = " << a-b << endl;
    cout << "a * b = " << a*b << endl;
    cout << "a / b = " << a/b << endl;    // integer division
    cout << "a % b = " << a%b << endl;    // remainder
    cout << "a > b is: " << (a>b) << endl; // 1 = true
    cout << (a>0 && b>0 ? "Both positive" : "Not both") << endl;
    return 0;
}
```

Output:

```
a + b = 13
a - b = 7
a * b = 30
a / b = 3
a % b = 1
a > b is: 1
Both positive
```

UNIT 2: Control Structures, Arrays & Strings

2.1 Control Structures

Control structures decide the flow of execution of a program. Without control structures, a program executes line by line from top to bottom. Using control structures, we can make decisions, repeat actions, and choose different paths.

Control Structures Overview

Types of Control Structures:

1. Sequential — Default, line by line
2. Decision Making — if, if-else, if-else-if, switch
3. Looping — for, while, do-while
4. Jump Statements — break, continue, goto, return

2.2 Decision Making Statements

2.2.1 if Statement

The if statement executes a block of code only when the given condition is true. If the condition is false, the block is skipped.

Syntax:

```
if (condition) {  
    // code executes when condition is TRUE  
}
```

2.2.2 if-else Statement

The if-else statement provides two paths — one when condition is true, another when it is false.

```
// if_else_example.cpp  
#include<iostream>  
using namespace std;  
int main() {  
    int marks;  
    cout << "Enter marks: ";  
    cin >> marks;  
    if(marks >= 75){  
        cout << "Result: DISTINCTION" << endl;  
    } else if(marks >= 60){  
        cout << "Result: FIRST CLASS" << endl;  
    } else if(marks >= 50){  
        cout << "Result: SECOND CLASS" << endl;  
    } else if(marks >= 35){  
        cout << "Result: PASS" << endl;  
    } else {  
        cout << "Result: FAIL" << endl;  
    }  
}
```

```
    }  
    return 0;  
}
```

Output:

```
Enter marks: 78  
Result: DISTINCTION
```

2.2.3 Switch Statement

The switch statement is used when we have multiple choices based on the value of a single variable or expression. It is more readable than multiple if-else-if conditions.

Important Rules: Each case must end with break; otherwise, all cases below it will also execute (called fall-through). The default case executes when no case matches.

```
// switch_example.cpp  
#include<iostream>  
using namespace std;  
int main() {  
    int day;  
    cout << "Enter day number (1-7): ";  
    cin >> day;  
    switch(day) {  
        case 1: cout << "Monday";    break;  
        case 2: cout << "Tuesday";   break;  
        case 3: cout << "Wednesday"; break;  
        case 4: cout << "Thursday";  break;  
        case 5: cout << "Friday";     break;  
        case 6: cout << "Saturday";   break;  
        case 7: cout << "Sunday";     break;  
        default: cout << "Invalid day!";  
    }  
    return 0;  
}
```

Output:

```
Enter day number (1-7): 5  
Friday
```

2.3 Loop Statements

Loops are used to execute a block of code repeatedly as long as a condition is true. This avoids writing the same code multiple times.

2.3.1 for Loop

The for loop is used when the number of iterations (repetitions) is known in advance.

Syntax: `for(initialization; condition; update){ body }`

```
// for_loop_example.cpp  
#include<iostream>  
using namespace std;  
int main() {  
    // Print multiplication table of 5  
    int n = 5;
```

```

    cout << "Multiplication Table of " << n << ":\n";
    for(int i = 1; i <= 10; i++){
        cout << n << " x " << i << " = " << n*i << endl;
    }
    return 0;
}

```

Output:

```

Multiplication Table of 5:
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
... (up to 5 x 10 = 50)

```

2.3.2 while Loop

The while loop is used when the number of iterations is NOT known in advance. It checks the condition before executing the body.

```

// while_loop_example.cpp
#include<iostream>
using namespace std;
int main(){
    // Find sum of digits of a number
    int n, sum = 0, digit;
    cout << "Enter a number: ";
    cin >> n;
    while(n > 0){
        digit = n % 10;    // get last digit
        sum = sum + digit;
        n = n / 10;        // remove last digit
    }
    cout << "Sum of digits = " << sum << endl;
    return 0;
}

```

Output:

```

Enter a number: 1234
Sum of digits = 10

```

2.3.3 do-while Loop

The do-while loop executes the body at least ONCE, then checks the condition. It is used in menu-driven programs where user must see the menu at least once.

```

// do_while_example.cpp
#include<iostream>
using namespace std;
int main(){
    int choice;
    do{
        cout << "\n--- MENU ---" << endl;
        cout << "1. Add" << endl;
        cout << "2. Subtract" << endl;
    }
}

```

```

        cout << "3. Exit" << endl;
        cout << "Enter choice: ";
        cin >> choice;
        if(choice == 1) cout << "You chose Add\n";
        else if(choice == 2) cout << "You chose Subtract\n";
    } while(choice != 3);
    cout << "Program ended.";
    return 0;
}

```

Output:

```

--- MENU ---
1. Add  2. Subtract  3. Exit
Enter choice: 1
You chose Add
Enter choice: 3
Program ended.

```

Comparison: for vs while vs do-while

Feature	for Loop	while Loop	do-while Loop
When to use	Known iterations	Unknown iterations	Execute at least once
Condition check	Before body	Before body	After body
Minimum execution	0 times possible	0 times possible	Always 1 time
Best used for	Counting, tables, arrays	Input validation	Menu-driven programs

2.4 Arrays

An array is a collection of elements of the SAME data type stored in CONTIGUOUS (adjacent) memory locations. All elements share the same name but are accessed using an index number.

Array Properties

Key Properties of Arrays:

- Fixed size: Must declare size at time of declaration
- Same data type: All elements must be of same type
- Index starts from 0: First element is at index 0
- Contiguous memory: All elements stored one after another
- Direct access: Can access any element directly using index

Syntax: `data_type array_name[size];`

Example: `int marks[5] = {85, 90, 78, 92, 88};`

One-Dimensional Array Program:

```
// array_1d.cpp
```

```
#include<iostream>
using namespace std;
int main(){
    int marks[5] = {85, 90, 78, 92, 88};
    int sum = 0;
    float avg;
    cout << "Student Marks:" << endl;
    for(int i = 0; i < 5; i++){
        cout << "Student " << i+1 << ": " << marks[i] << endl;
        sum += marks[i];
    }
    avg = (float)sum / 5;
    cout << "Total: " << sum << endl;
    cout << "Average: " << avg << endl;
    return 0;
}
```

Output:

```
Student Marks:
Student 1: 85
Student 2: 90
Student 3: 78
Student 4: 92
Student 5: 88
Total: 433
Average: 86.6
```

Two-Dimensional Array (Matrix):

A 2D array is like a table with rows and columns. It is also called a matrix.

Syntax: `data_type array_name[rows][columns];`

```
// array_2d.cpp
#include<iostream>
using namespace std;
int main(){
    int matrix[2][3] = {{1,2,3},{4,5,6}};
    cout << "Matrix:" << endl;
    for(int i = 0; i < 2; i++){
        for(int j = 0; j < 3; j++){
            cout << matrix[i][j] << "\t";
        }
        cout << endl;
    }
    return 0;
}
```

Output:

```
Matrix:
1      2      3
4      5      6
```

2.5 Strings in C++

A string is a sequence of characters. In C++, strings can be handled in two ways: as character arrays (C-style strings) or using the string class (C++ style).

String Functions (Important for Exam):

Function	Purpose	Example	Output
length() / size()	Returns length of string	s.length()	5 (for "Hello")
append()	Adds string at end	s.append(" World")	"Hello World"
substr(pos,len)	Extracts substring	s.substr(0,3)	"Hel"
find(str)	Finds position of substring	s.find("lo")	3
compare()	Compares two strings	s.compare("Hello")	0 if equal
at(index)	Character at position	s.at(1)	'e'
empty()	Checks if string is empty	s.empty()	0 (false)
reverse	Reverse a string	reverse(s.begin(),s.end())	"olleH"

```
// string_functions.cpp
#include<iostream>
#include<string>
using namespace std;
int main() {
    string s = "Hello";
    cout << "String: " << s << endl;
    cout << "Length: " << s.length() << endl;
    s.append(" World");
    cout << "After append: " << s << endl;
    cout << "Substring(0,5): " << s.substr(0,5) << endl;
    cout << "Find 'World': position " << s.find("World") << endl;
    if(s.compare("Hello World") == 0)
        cout << "Strings are equal" << endl;
    return 0;
}
```

Output:

```
String: Hello
Length: 5
After append: Hello World
Substring(0,5): Hello
Find 'World': position 6
Strings are equal
```

UNIT 3: Functions, Structures & Unions

3.1 Functions in C++

A function is a block of code that performs a specific task. It can be called (executed) multiple times from anywhere in the program. Functions help in code reusability, modularity, and readability.

Why Use Functions?

Advantages of Functions:

1. Code Reusability: Write once, call many times
2. Modularity: Break large program into small parts
3. Easy Debugging: Easy to find and fix errors
4. Readability: Program becomes easier to understand
5. Reduces code length and duplication

Function Components:

Component	Description	Example
Return Type	Data type of value returned (void if nothing returned)	int, float, void
Function Name	Name to identify the function	add, display, calculate
Parameters	Input values passed to function (optional)	(int a, int b)
Function Body	Block of code that executes when function is called	{ return a+b; }
Function Call	Statement that invokes/executes the function	result = add(5, 3);
Return Statement	Sends value back to calling function	return a+b;

3.2 Parameter Passing Methods

When a function is called, we can pass values to it in three ways. This is one of the MOST IMPORTANT topics for exams.

3.2.1 Pass by Value

In pass by value, a COPY of the actual argument is passed to the function. Any changes made inside the function do NOT affect the original variable. The original variable remains safe.

```
// pass_by_value.cpp
#include<iostream>
using namespace std;

void addTen(int x){           // x is a copy of 'a'
    x = x + 10;               // only copy changes
```

```

        cout << "Inside function: x = " << x << endl;
    }

    int main(){
        int a = 5;
        cout << "Before call: a = " << a << endl;
        addTen(a);           // passes copy of a
        cout << "After call:  a = " << a << endl; // unchanged
        return 0;
    }

```

Output:

```

Before call: a = 5
Inside function: x = 15
After call:  a = 5 // Original not changed!

```

3.2.2 Pass by Reference

In pass by reference, the function receives a REFERENCE (alias) to the actual variable. Changes made inside the function directly affect the original variable. The & symbol is used.

```

// pass_by_reference.cpp
#include<iostream>
using namespace std;

void swap(int &x, int &y){ // & means reference
    int temp = x;
    x = y;
    y = temp;
}

int main(){
    int a = 10, b = 20;
    cout << "Before swap: a=" << a << ", b=" << b << endl;
    swap(a, b);           // passes reference
    cout << "After swap:  a=" << a << ", b=" << b << endl;
    return 0;
}

```

Output:

```

Before swap: a=10, b=20
After swap:  a=20, b=10 // Values swapped!

```

3.2.3 Pass by Address (Pointer)

In pass by address, the address (memory location) of the actual variable is passed using pointers. The * operator is used inside the function to access and modify the original value.

```

// pass_by_address.cpp
#include<iostream>
using namespace std;

void doubleIt(int *ptr){ // ptr holds address of variable
    *ptr = (*ptr) * 2;    // * dereferences to get actual value
}

```

```
int main(){
    int a = 7;
    cout << "Before: a = " << a << endl;
    doubleIt(&a);          // pass address of a using &
    cout << "After:  a = " << a << endl;
    return 0;
}
```

Output:

```
Before: a = 7
After:  a = 14
```

Comparison of All 3 Parameter Passing Methods:

Feature	By Value	By Reference	By Address
Symbol used	No special symbol	& (ampersand)	* and & (pointer)
What is passed?	Copy of value	Alias of variable	Memory address
Original changed?	NO	YES	YES
Memory	Extra memory for copy	No extra memory	Memory for pointer
Speed	Slower for large data	Faster	Faster
Use case	When original must be safe	Swap, modify multiple values	Dynamic memory, arrays

3.3 Inline Functions

An inline function is a function where the function call is replaced by the function body itself at the point of the call. This saves the overhead of a function call and makes the program faster.

The inline keyword is a request to the compiler — the compiler may or may not honor it. Inline functions should be small (1-3 lines). Large functions with loops are not suitable for inline.

```
// inline_function.cpp
#include<iostream>
using namespace std;

inline int square(int x){
    return x * x;
}

inline float circleArea(float r){
    return 3.14159 * r * r;
}

int main(){
    cout << "Square of 6: " << square(6) << endl;
    cout << "Square of 9: " << square(9) << endl;
    cout << "Area of circle (r=5): " << circleArea(5) << endl;
    return 0;
}
```

Output:

```
Square of 6: 36
Square of 9: 81
Area of circle (r=5): 78.5397
```

3.4 Recursive Functions

A recursive function is a function that calls ITSELF. Every recursive function must have a BASE CASE (termination condition) to stop the recursion. Without a base case, the function will call itself infinitely (infinite recursion / stack overflow).

```
// recursive_function.cpp
#include<iostream>
using namespace std;

// Factorial using recursion: n! = n * (n-1)!
int factorial(int n){
    if(n == 0 || n == 1)    // BASE CASE
        return 1;
    return n * factorial(n-1); // RECURSIVE CALL
}

// Fibonacci using recursion
int fibonacci(int n){
    if(n == 0) return 0;    // BASE CASE 1
    if(n == 1) return 1;    // BASE CASE 2
    return fibonacci(n-1) + fibonacci(n-2); // RECURSIVE
}

int main(){
    cout << "Factorial of 5 = " << factorial(5) << endl;
    cout << "Factorial of 7 = " << factorial(7) << endl;
    cout << "Fibonacci(8) = " << fibonacci(8) << endl;
    return 0;
}
```

Output:

```
Factorial of 5 = 120
Factorial of 7 = 5040
Fibonacci(8) = 21
```

3.5 Structures

A structure is a user-defined data type that groups together variables of DIFFERENT data types under one name. It is used to represent real-world entities like Student, Employee, Product, etc.

Syntax:

```
struct StructureName {
    data_type member1;
    data_type member2;
    ... (can have any number of members)
};
```

```
// structure_example.cpp
#include<iostream>
#include<string>
using namespace std;

struct Student {
    int roll;
    string name;
    float marks;
    char grade;
};

int main(){
    Student s1;    // declare structure variable
    s1.roll = 101;
    s1.name = "Vishal";
    s1.marks = 87.5;
    s1.grade = 'A';

    cout << "Roll No: " << s1.roll << endl;
    cout << "Name: " << s1.name << endl;
    cout << "Marks: " << s1.marks << endl;
    cout << "Grade: " << s1.grade << endl;
    return 0;
}
```

Output:

```
Roll No: 101
Name: Vishal
Marks: 87.5
Grade: A
```

3.6 Unions

A union is also a user-defined data type similar to a structure. The key difference is that all members of a union SHARE THE SAME MEMORY LOCATION. Only one member can hold a value at a time. The size of a union equals the size of its LARGEST member.

```
// union_example.cpp
#include<iostream>
using namespace std;

union Data {
    int i;
    float f;
    char c;
};

int main(){
    Data d;
    d.i = 10;
    cout << "d.i = " << d.i << endl;

    d.f = 3.14;    // overwrites d.i in memory
    cout << "d.f = " << d.f << endl;
    cout << "d.i now = " << d.i << endl;    // garbage!
}
```

```

    cout << "Size of union: " << sizeof(d) << " bytes" << endl;
    return 0;
}

```

Output:

```

d.i = 10
d.f = 3.14
d.i now = 1078523331 // garbage value!
Size of union: 4 bytes

```

Structure vs Union — Complete Comparison (Exam Important):

Feature	Structure	Union
Keyword	struct	union
Memory Allocation	Separate memory for each member	All members share same memory
Total Size	Sum of sizes of all members	Size of the LARGEST member only
Members at once	All members can hold values simultaneously	Only ONE member can be used at a time
Data safety	Data of all members safe	Storing new value destroys old value
Example use	Student record, Employee data	Memory saving, type punning
Example	struct Student{int r; float m;}; // size=8	union Data{int i; float f;}; // size=4

UNIT 4: Pointers

4.1 Introduction to Pointers

A pointer is a special variable that stores the MEMORY ADDRESS of another variable. Instead of storing a data value directly, it stores where that data is located in computer memory.

Pointer Fundamentals

Key Pointer Concepts:

- Every variable occupies memory — memory has an address
- & operator: gives the ADDRESS of a variable
- * operator (dereference): gives the VALUE at an address
- Pointer variable: stores an address, not a direct value
- Pointer must be declared with the same type as the variable it points to

Syntax: `data_type *pointer_name;`

Example: `int *p;` (p is a pointer to int)

`float *fp;` (fp is a pointer to float)

4.2 Address Variable & Pointer Variable

```
// pointer_basics.cpp
#include<iostream>
using namespace std;
int main() {
    int a = 50;           // regular variable
    int *p;               // pointer variable
    p = &a;               // p stores address of a

    cout << "Value of a:      " << a << endl;
    cout << "Address of a (&a): " << &a << endl;
    cout << "Value of p:      " << p << endl; // same as &a
    cout << "Value at p (*p):  " << *p << endl; // same as a

    *p = 100;             // change value through pointer
    cout << "After *p=100, a = " << a << endl;
    return 0;
}
```

Output:

```
Value of a:      50
Address of a (&a): 0x61ff08 (actual address may differ)
Value of p:      0x61ff08 (same address)
Value at p (*p): 50
After *p=100, a = 100
```

Pointer Operators Table:

Operator	Name	Purpose	Example
&	Address-of operator	Gets memory address of a variable	&a gives address of a
*	Dereference operator	Gets value stored at an address	*p gives value at address p
->	Arrow operator	Access members through pointer to struct	p->name (OOP use)

4.3 Pointer Arithmetic

We can perform arithmetic operations on pointers. When we increment a pointer, it moves to the NEXT memory location of the SAME DATA TYPE (not just +1 byte, but +sizeof(type) bytes).

```
// pointer_arithmetic.cpp
#include<iostream>
using namespace std;
int main(){
    int arr[4] = {10, 20, 30, 40};
    int *p = arr;          // p points to arr[0]

    cout << "Using pointer to traverse array:" << endl;
    for(int i = 0; i < 4; i++){
        cout << "arr[" << i << "] = " << *p << "   Address: " << p <<
endl;
        p++;                // moves to next integer (4 bytes ahead)
    }
    return 0;
}
```

Output:

```
Using pointer to traverse array:
arr[0] = 10   Address: 0x61ff00
arr[1] = 20   Address: 0x61ff04   (4 bytes ahead)
arr[2] = 30   Address: 0x61ff08
arr[3] = 40   Address: 0x61ff0c
```

4.4 Uses of Pointers

- Dynamic Memory Allocation: Allocate memory at runtime using new and delete
- Pass by Address: Modify original variable values from inside a function
- Array Handling: Arrays and pointers are closely related in C++
- String Handling: Char pointers for C-style strings
- Data Structures: Linked lists, trees, graphs use pointers extensively
- Function Pointers: Pointer to a function (advanced use)

Dynamic Memory Allocation with new and delete:

```
// dynamic_memory.cpp
#include<iostream>
```

```
using namespace std;
int main(){
    // Allocate single integer dynamically
    int *p = new int;      // allocates 4 bytes
    *p = 99;
    cout << "Dynamic value: " << *p << endl;
    delete p;              // free memory

    // Allocate dynamic array
    int n = 5;
    int *arr = new int[n];
    for(int i = 0; i < n; i++)
        arr[i] = (i+1) * 10;
    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
    delete[] arr;          // free array memory
    return 0;
}
```

Output:

```
Dynamic value: 99
10 20 30 40 50
```

NULL Pointer:

A pointer that does not point to any valid memory location is called a NULL pointer. It is good practice to initialize pointers to NULL when not in use to avoid dangling pointer errors.

```
int *p = NULL;      // or int *p = nullptr; (C++11)
if(p == NULL){
    cout << "Pointer is NULL, not pointing anywhere";
}
```

Common Pointer Errors (Exam Tip)

Common Pointer Mistakes to Avoid:

1. Uninitialized pointer: `int *p; *p=5; // DANGEROUS!`
2. Dangling pointer: Using pointer after delete
3. Memory leak: Forgetting to use delete after new
4. NULL dereference: `*p` when `p` is NULL -> crash
5. Type mismatch: `float *p = &intVar; // wrong!`

Quick Revision Summary

Unit	Key Topics	Must-Know Programs
Unit 1 - Basics	Tokens (5 types), Keywords, Identifiers, Variables, Data Types, Operators, Precedence	Operators program, Variable demo
Unit 2 - Control	if/else/switch, for/while/do-while, 1D/2D Arrays, String functions	Grade calculator, Menu program, Array average
Unit 3 - Functions	Pass by value/reference/address, inline, recursive, struct vs union	Swap, Factorial, Student record
Unit 4 - Pointers	& and * operators, pointer arithmetic, dynamic memory, NULL pointer	Pointer basics, Array via pointer, new/delete